

Tipo Estrutura



Objetivos do Tópico

■ Objetivos do tópico:

- Compreender o conceito e aplicação de tipos estruturados.
- Declarar tipos estruturados.
- Empregar tipos estruturados para resolver problemas.

Tipos Estruturados

Tipos de dados básicos:

- int , char, float etc. armazenam apenas uma informação.

Tipo estrutura:

- Permite agrupar dados que por outro lado ficariam dispersos no programa.
- Pode-se agrupar dados de diferentes tipos.

Por exemplo:

- Se você precisar armazenar os dados de um aluno : nome, idade e renda, estes ficarão separados no código quando utilizamos apenas dados básicos.

Tipos Estruturados x Tipos Básicos

```
#include <stdio.h>

int main(){
    char nome[10];
    int idade;
    float renda;
    printf("Informe o nome: ");
    scanf("%s", &nome);
    printf("Informe a idade: ");
    scanf("%d", &idade);
    printf("Informe a renda: ");
    scanf("%f", &renda);
    printf("\nNome : %s", nome);
    printf("\nIdade: %d", idade);
    printf("\nRenda: %.2f", renda);
    return 0;
}
```

As variáveis são declaradas separadamente e não indicam claramente que se tratam de informações sobre um aluno, ou seja, não está claro que: nome, idade e renda referem-se a um aluno.

Tipos Estruturados x Tipos Básicos

```
struct aluno {  
    char nome[10];  
    int idade;  
    float renda;  
};
```

Por meio de um tipo estruturado é possível agrupar variáveis que ficariam dispersas no programa.

```
main() {  
    struct aluno aluno1;  
    struct aluno aluno2;  
    printf("Informe o nome: ");  
    scanf("%s", &aluno1.nome);  
    printf("Informe a idade: ");  
    scanf("%d", &aluno1.idade);  
    printf("Informe a renda: ");  
    scanf("%f", &aluno1.renda);  
    printf("\nNome : %s ", aluno1.nome);  
    printf("\nIdade: %d ", aluno1.idade);  
    printf("\nRenda: %.2f ", aluno1.renda);  
}
```

O tipo estruturado é como um tipo de dado, você poderá criar diversas variáveis a partir desse tipo.

As estruturas são declaradas antes da função main.

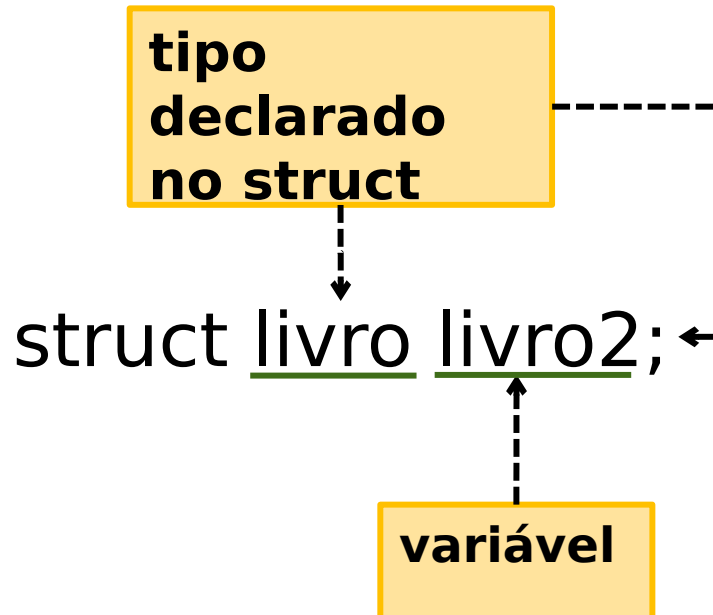
Declaração de Tipos Estruturados

```
1  struct livro {
2      char titulo[30];
3      int paginas;
4      float preco;
5  };
6
7  struct {
8      char titulo[30];
9      int paginas;
10     float preco;
11 } livro1, livro2;
12
13 typedef struct {
14     char titulo[30];
15     int paginas;
16     float preco;
17 } LIVRO;
```

Qual a diferença entre as declarações de estrutura?

Declaração de Tipos Estruturados

**Declaração mais comum.
Apenas declara a estrutura.
A variável “tipo estrutura” é
declarada no código.**



```
1 struct livro {  
2     char titulo[30];  
3     int paginas;  
4     float preco;  
5 };  
6  
7 struct {  
8     char titulo[30];  
9     int paginas;  
10    float preco;  
11 } livro1, livro2;  
12  
13 typedef struct {  
14     char titulo[30];  
15     int paginas;  
16     float preco;  
17 } LIVRO;  
18  
19 int main() {  
20     livro1.paginas = 1;  
21     struct livro livro2;  
22     LIVRO livro4;  
23     return 0;  
24 }  
25
```

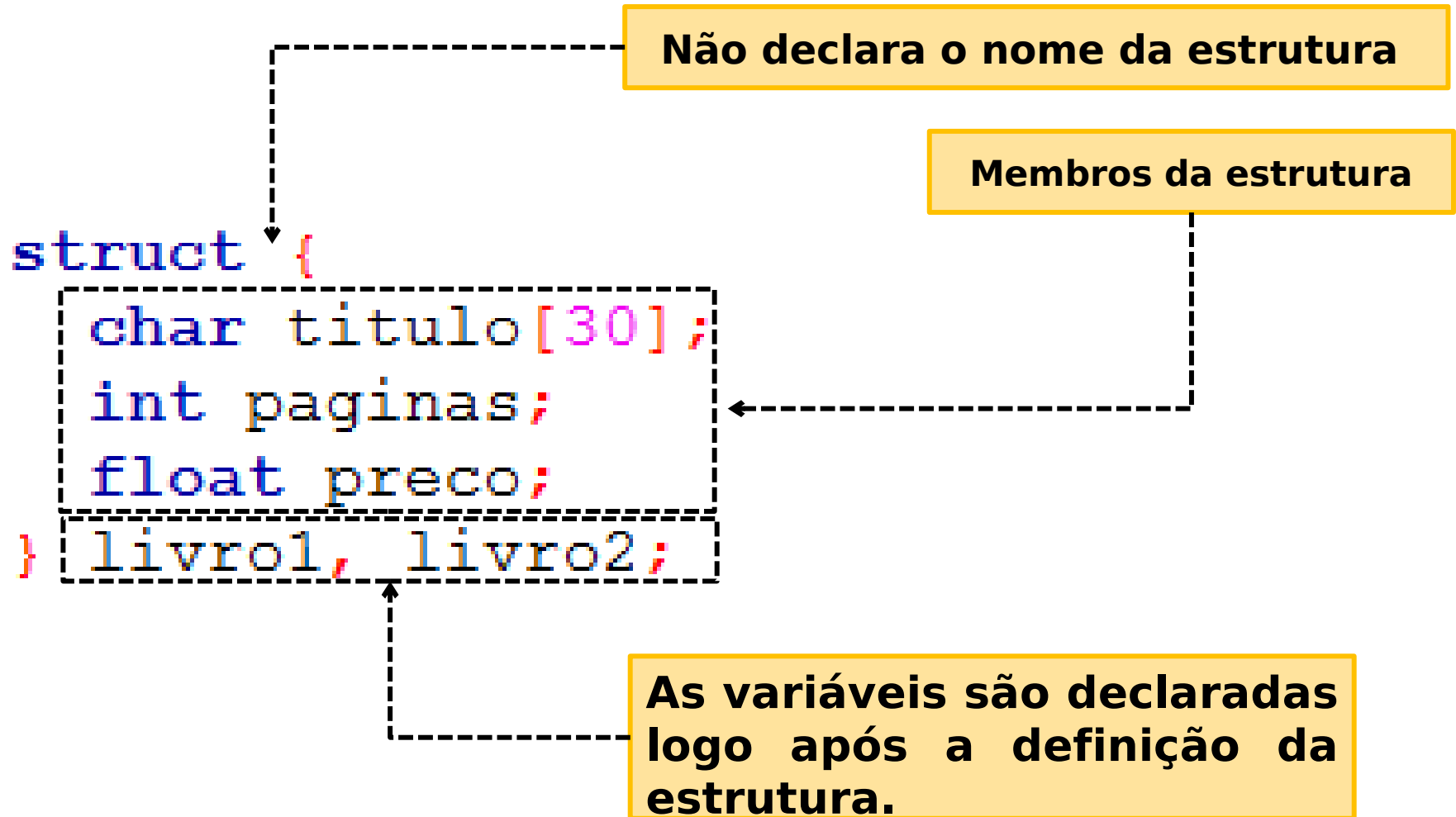
Declaração de Tipos Estruturados

Nome da estrutura

```
struct livro {  
    char titulo[30];  
    int paginas;  
    float preco;  
};
```

Membros da estrutura

Declaração de Tipos Estruturados



Declaração de Tipos Estruturados

Nesse caso, além da declaração do tipo estruturado, também é declarada a variável (ou variáveis).

livro1.paginas = 1;

Note, que nesse caso, a variável já pode ser utilizada diretamente no código pois já foi declarada junto ao tipo.

```
1 struct livro {
2     char titulo[30];
3     int paginas;
4     float preco;
5 };
6
7 struct {
8     char titulo[30];
9     int paginas;
10    float preco;
11 } livro1, livro2;
12
13 typedef struct {
14     char titulo[30];
15     int paginas;
16     float preco;
17 } LIVRO;
18
19 int main(){
20     livro1.paginas = 1;
21     struct livro livro2;
22     LIVRO livro4;
23     return 0;
24 }
25
```

Declaração de Tipos Estruturados

LIVRO livro4; ←

As variáveis são declaradas apenas informando o nome do identificador da estrutura, como uma declaração comum.

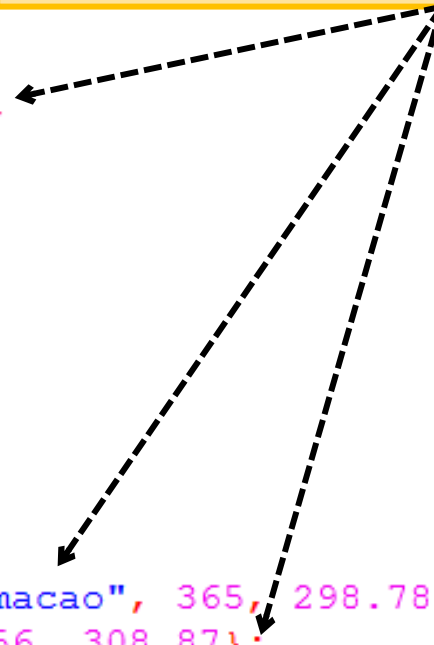
Nessa declaração typedef define que LIVRO é um sinônimo da estrutura.

```
1 struct livro {
2     char titulo[30];
3     int paginas;
4     float preco;
5 };
6
7 struct {
8     char titulo[30];
9     int paginas;
10    float preco;
11 } livro1, livro2;
12
13 typedef struct {
14     char titulo[30];
15     int paginas;
16     float preco;
17 } LIVRO;
18
19 int main() {
20     livro1.paginas = 1;
21     struct livro livro2;
22     LIVRO livro4;
23     return 0;
24 }
25
```

Inicialização de Variáveis Tipo Estrutura

```
1 struct livro {
2     char titulo[30];
3     int paginas;
4     float preco;
5 };
6
7 struct {
8     char titulo[30];
9     int paginas;
10    float preco;
11 } livro1 = {"Redes", 278, 79.87},
12    livro2;
13
14 typedef struct {
15     char titulo[30];
16     int paginas;
17     float preco;
18 } LIVRO;
19
20 int main(){
21     livro1.paginas = 1;
22     struct livro livro2 = {"Programacao", 365, 298.78};
23     LIVRO livro4 = {"Estrutura", 456, 308.87};
24     return 0;
25 }
26
```

Variáveis criadas a partir de tipos estruturados também podem ser declaradas com valores iniciais.



Membros da Estrutura

- Como uma variável declarada com estrutura (também chamada de registro) possui membros (ou campos), é possível acessar esses membros

```
struct telefone {  
    char ddd[2];  
    int numero;  
};
```

Declaração da estrutura

Declaração da variável

```
main() {  
    struct telefone fone;
```

```
    printf("Informe o DDD: ");  
    scanf("%s", &fone.ddd);
```

Leitura do elemento

```
    printf("Informe o Numero:");  
    scanf("%i", &fone.numero);
```

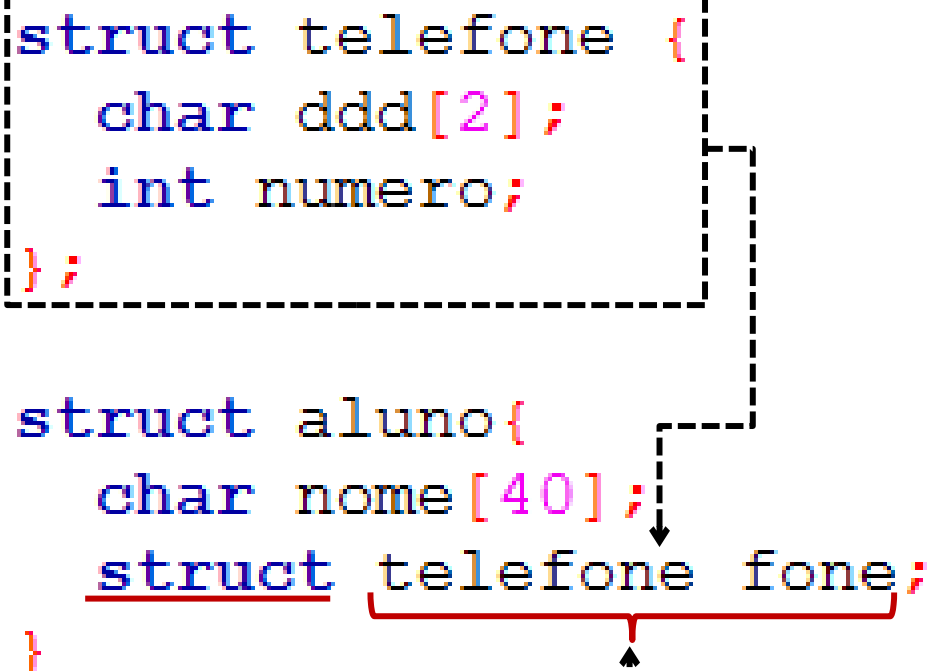
Saída do elemento

```
    printf("Numero Telefone: (%s) %i", fone.ddd, fone.numero);  
}
```

Estrutura Contidos em Outras Estruturas

- É possível declarar um membro (campo) de uma estrutura como uma estrutura:

```
struct telefone {  
    char ddd[2];  
    int numero;  
};  
  
struct aluno{  
    char nome[40];  
    struct telefone fone;  
}
```



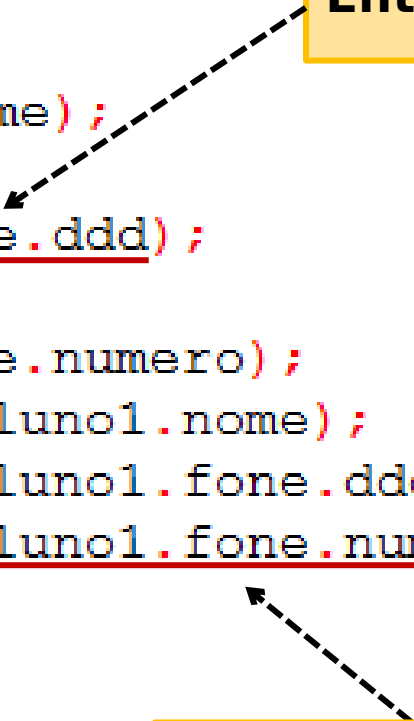
Declaração do membro de outra estrutura como uma variável tipo estrutura. Observe que a palavra 'struct' é utilizada na declaração.

Estrutura Contidos em Outras Estruturas

- A gravação e acesso aos membros de uma variável tipo estrutura obedece a mesma lógica.

```
main() {  
    struct aluno aluno1;  
    printf("Nome: ");  
    scanf("%s", &aluno1.nome);  
    printf("DDD : ");  
    scanf("%s", &aluno1.fone.ddd);  
    printf("Fone: ");  
    scanf("%i", &aluno1.fone.numero);  
    printf("\nNome: %s", aluno1.nome);  
    printf("\nDDD : %s", aluno1.fone.ddd);  
    printf("\nFone: %i", aluno1.fone.numero);  
}
```

Entrada de dados



Saída de dados

Comparando Tipos Estruturas

```
struct janela2D {  
    int largura;  
    int altura;  
};
```

Definição e inicialização
de duas variáveis tipo
estrutura

```
int main() {  
    struct janela2D j1 = {10, 20};  
    struct janela2D j2 = {30, 20};  
  
    if (j1 == j2)  
        printf("Mesmas dimensoes");  
    else printf("Dimensoes diferentes");  
    return 0;  
}
```

Errado

Embora, aparentemente pareça correto comparar as duas variáveis diretamente, essa sintaxe é INCORRETA. Não podemos comparar a variável mas sim os elementos. **Veja o próximo slide.**

Comparando Tipos Estruturas

```
struct janela2D {  
    int largura;  
    int altura;  
};
```

**Definição e
inicialização de
duas variáveis tipo
estrutura**

```
int main() {
```

```
    struct janela2D j1 = {20,10};  
    struct janela2D j2 = {10,10};
```

```
    if (j1.altura == j2.altura)  
        if (j1.largura == j2.largura)  
            printf("Mesmas dimensoes");  
        else printf("Larguras diferentes");  
    else printf("Alturas diferentes");  
    return 0;
```

```
}
```

Certo

**Para comparar tipos estrutura é necessário
comparar cada elemento que compõe a
estrutura.**



Estruturas e Ponteiros

```
struct janela2D {  
    int largura;  
    int altura;  
};
```

```
int main() {  
    struct janela2D j1 = {11,12};  
    struct janela2D j2 = {11,12};  
    struct janela2D *j;  
    j = &j2;  
  
    if (j1.altura == (*j).altura)  
        if (j1.largura == (*j).largura)  
            printf("Mesmas dimensoes");  
        else printf("Larguras diferentes");  
    else printf("Alturas diferentes");  
    return 0;  
}
```

Estruturas também podem ser declaradas e endereçadas como ponteiros.

Porém, deve-se empregar os parênteses para assegurar maior precedência ao operador asterisco (*) ante o operador ponto (.).

Estrutura como Parâmetro (por Valor)

```
struct quadrado {  
    int largura;  
    int altura;  
};
```

Declaração GLOBAL
da estrutura

Escopo da função
declarando a
estrutura como
parâmetro
(passagem por
valor)

```
int calcularArea (int l, int a);
```

```
int main(){  
    struct quadrado meuQuadrado;  
    printf("Informe a largura: ");  
    scanf("%i", &meuQuadrado.largura);  
    printf("Informe a altura : ");  
    scanf("%i", &meuQuadrado.altura);  
    int area = calcularArea(meuQuadrado.altura,  
                            meuQuadrado.largura);  
    printf("Area do quadrado : %d\n", area);  
    return 0;  
}
```

Declaração da
variável tipo
estrutura

Chamada da
função (valor)

```
int calcularArea (int l, int a){  
    return l * a;  
}
```

Estrutura como Parâmetro (por Referência)

```
struct quadrado {  
    int largura;  
    int altura;  
};
```

Declaração GLOBAL
da estrutura

Escopo da função
declarando a
estrutura como
parâmetro
(**passagem por
referência**)

```
int calcularArea (int *l, int *a)
```

```
int main() {  
    struct quadrado meuQuadrado;  
    printf("Informe a largura: ");  
    scanf("%i", &meuQuadrado.largura);  
    printf("Informe a altura : ");  
    scanf("%i", &meuQuadrado.altura);  
    int area = calcularArea(&meuQuadrado.altura,  
                           &meuQuadrado.largura);  
    printf("Area do quadrado : %d\n", area);  
    return 0;  
}
```

Declaração da
variável tipo
estrutura

Chamada da
função (**endereço**)

```
int calcularArea (int *l, int *a) {  
    return *l * *a;  
}
```

Estruturas como Parâmetro (por Valor)

```
struct quadrado {  
    int largura;  
    int altura;  
};
```

Declaração GLOBAL
da estrutura

Escopo da função
declarando a
estrutura como
parâmetro
(passagem por
valor)

```
int calcularArea (struct quadrado q);
```

```
int main() {  
    struct quadrado meuQuadrado;  
    printf("Informe a largura: ");  
    scanf("%i", &meuQuadrado.largura);  
    printf("Informe a altura : ");  
    scanf("%i", &meuQuadrado.altura);  
    int area = calcularArea(meuQuadrado);  
    printf("Area do quadrado : %d\n", area);  
    return 0;  
}
```

Declaração da
variável tipo
estrutura

Chamada da função
(conteúdo da
variável)

```
int calcularArea (struct quadrado q) {  
    return q.altura * q.largura;  
}
```

Estrutura como Parâmetro (por Referência)

```
struct quadrado {  
    int largura;  
    int altura;  
};
```

Declaração GLOBAL
da estrutura

Escopo da
função
declarando a
estrutura como
parâmetro
(**passagem por
referência**)

```
int calcularArea (struct quadrado *q);
```

```
int main() {  
    struct quadrado meuQuadrado;  
    printf("Informe a largura: ");  
    scanf("%i", &meuQuadrado.largura);  
    printf("Informe a altura : ");  
    scanf("%i", &meuQuadrado.altura);  
    int area = calcularArea(&meuQuadrado);  
    printf("Area do quadrado : %d\n", area);  
    return 0;  
}
```

Declaração da
variável tipo
estrutura

Chamada da
função (**endereço**)

```
int calcularArea (struct quadrado *q) {  
    return (*q).altura * (*q).largura;  
}
```

Referências Bibliográficas

-  [01] TENENBAUM, Aaron M.; LANGSAM, Yedidyah; AUGENSTEIN, Moshe. Estruturas de dados usando C. São Paulo: Makron, 1995.
-  [02] CELES, Waldemar; CERQUEIRA, Renato Fontoura de Gusmão; RANGEL NETTO, José Lucas Mourão. Introdução a estrutura de dados: com técnicas de programação em C. Rio de Janeiro, RJ: Elsevier, 2004.
-  [03] PINHEIRO, F. de A. C. Elementos de Programação em C. Editora Bookman, 2012.

Passagem de Vetores como Parâmetros

■ Segundo Celes et al ([2], p. 62):

- Passar um vetor como parâmetro consiste em passar o endereço da primeira posição do vetor.
- Logo, um vetor (endereço inicial) é passado por referência e não por valor.
- Na declaração da assinatura da função (escopo ou protótipo) deve ser declarado um parâmetro do tipo ponteiro.

Passagem de Vetores como Parâmetros

```
void lerVetor (int tamanhoVetor, int *vetor);
void mostrarVetor (int tamanhoVetor, int vetor[]);

int main(){
    int nroProvas = 4;
    int nota[nroProvas];
    int contador;
    lerVetor(nroProvas, &nota);
    mostrarVetor(nroProvas, &nota);
    printf("\nMedia : %3d", (nota[0]+nota[1]+nota[2]+nota[3])/4);
    return 0;
}

void lerVetor (int tamanhoVetor, int *vetor){
    int contador;
    for (contador = 0; contador < tamanhoVetor; contador++){
        printf("Informe a %da nota: ", contador+1);
        scanf("%d", &vetor[contador]);
    }
}

void mostrarVetor (int tamanhoVetor, int vetor[]){
    int contador;
    for (contador = 0; contador < tamanhoVetor; contador++){
        printf("nota %d: %3d \n", contador+1, vetor[contador]);
    }
}
```

O endereço do vetor (primeira posição) é passado como parâmetro

O parâmetro que receberá o vetor pode ser declarado como um ponteiro ou...

...o parâmetro também pode ser declarado com os colchetes (sem o número de elementos)

Passagem de Vetores como Parâmetros

```
void lerVetor (int tamanhoVetor, int *vetor);
void mostrarVetor (int tamanhoVetor, int vetor[4]);

int main(){
    int nroProvas = 4;
    int nota[nroProvas];
    int contador;
    lerVetor(nroProvas, &nota);
    mostrarVetor(nroProvas, nota);
    printf("\nMedia : %3d", (nota[0]+nota[1]+nota[2]+nota[3])/4);
    return 0;
}

void lerVetor (int tamanhoVetor, int *vetor){
    int contador;
    for (contador = 0; contador < tamanhoVetor; contador++){
        printf("Informe a %da nota: ", contador+1);
        scanf("%d", &vetor[contador]);
    }
}

void mostrarVetor (int tamanhoVetor, int vetor[4]){
    int contador;
    for (contador = 0; contador < tamanhoVetor; contador++){
        printf("Nota %d.: %3d \n", contador+1, vetor[contador]);
    }
}
```

Mesmo que o parâmetro seja declarado como um vetor e tenha seu tamanho definido, ainda assim não é criada uma variável local do tipo vetor na função. A passagem ainda será por referência, ou seja, será passado o endereço do vetor e não uma cópia do vetor passado na chamada da função.

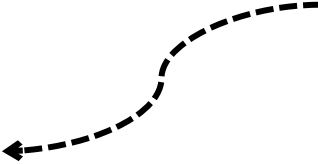
Vetor de Estrutura

As estruturas também podem ser declaradas em matrizes: matrizes de estrutura.

Primeiro declara-se a estrutura e depois a matriz.

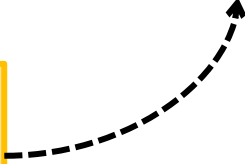
```
struct agendaEmail{  
    char nome[30];  
    char email[50];  
};
```

**Declaração da
estrutura**



```
struct agendaEmail minhaAgenda[10];
```

**Declaração da
matriz de
estrutura**

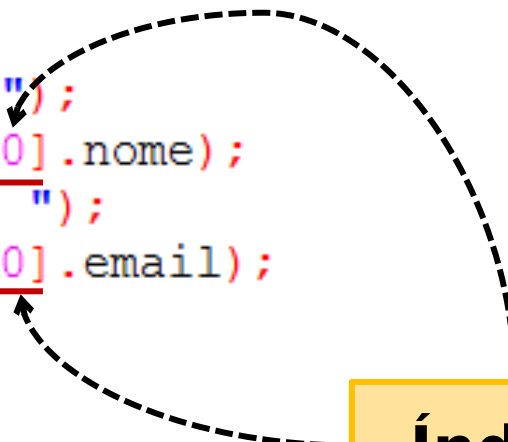


Vetor de Estrutura

- ❏ O acesso aos membros de uma matriz de estrutura se dá por meio de um índice logo após o nome da estrutura.

```
struct agendaEmail {  
    char nome[30];  
    char email[50];  
};
```

```
main() {  
    struct agendaEmail minhaAgenda[10];  
    int i;  
    for (i=0; i < 10; i++){  
        printf("Informe o nome: ");  
        scanf("%s", &minhaAgenda[0].nome);  
        printf("Informe o email: ");  
        scanf("%s", &minhaAgenda[0].email);  
    }  
}
```



Índice

Vetor de Estrutura: Exemplo Completo

```
struct agendaEmail{  
    char nome[30];  
    char email[50];  
};
```

Constante

```
#define MAXCONTATO 2
```

Entrada

```
main(){  
    struct agendaEmail minhaAgenda[MAXCONTATO];  
    int i;  
    for (i=0; i < MAXCONTATO; i++){  
        printf("Informe o nome: ");  
        scanf("%s",&minhaAgenda[i].nome);  
        printf("Informe o email: ");  
        scanf("%s",&minhaAgenda[i].email);  
    }
```

Saída

```
    for (i=0; i < MAXCONTATO; i++){  
        printf("Nome : %s\n", minhaAgenda[i].nome);  
        printf("Email: %s\n", minhaAgenda[i].email);  
    }  
}
```

Vetor de Estrutura como Parâmetro

```
struct Agenda {  
    char primeiroNome[30];  
    int ddd;  
    int numero;  
};
```

```
void adicionarContatos (int nroContatos, struct Agenda agenda[]);
```

```
int main () {  
    int totalContatos = 2;  
    struct Agenda agendaPessoal[totalContatos];  
    adicionarContatos(totalContatos, &agendaPessoal);  
    return 0;  
}
```

```
void adicionarContatos (int nroContatos, struct Agenda agenda[]) {  
    int contador;  
    for (contador = 0; contador < nroContatos; contador++) {  
        printf("Informe o primeiro nome: ");  
        scanf("%s", &(agenda)[contador].primeiroNome);  
    }  
}
```

Endereço do vetor de estrutura é passado como parâmetro para função

Leitura do membro "primeiroNome" do primeiro elemento do vetor "agenda"

Exercícios

```
struct Agenda {  
    char primeiroNome[30];  
    int ddd;  
    int numero;  
};
```

```
void adicionarContatos (int nroContatos, struct Agenda agenda[]);
```

```
int main () {  
    int totalContatos = 2;  
    struct Agenda agendaPessoal[totalContatos];  
    adicionarContatos(totalContatos, &agendaPessoal);  
    return 0;  
}
```

Crie uma nova função que receba o vetor como parâmetro e mostre todos os contatos cadastrados.

Termine a função adicionar contatos , leia o ddd e o número do telefone.

```
void adicionarContatos (int nroContatos, struct Agenda agenda[]) {  
    int contador;  
    for (contador = 0; contador < nroContatos; contador++) {  
        printf("Informe o primeiro nome: ");  
        scanf("%s", &(agenda)[contador].primeiroNome);  
    }  
}
```